

Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 «Программная инженерия» –

Системное и прикладное программное обеспечение

Дисциплина: Рефакторинг баз данных и приложений

Этап 3: Модульная архитектура

Выполнили:

студенты 4 курса

Кравцов Кирилл Денисович

Группа: Р3411

Батманов Даниил Евгеньевич

Группа: Р3407

Преподаватель:

Логинов Иван Павлович

г. Санкт-Петербург, 2025

3.1. Разделение на модули с Spring Modulith

[Pull request](#)

Задача

Проблема: Монолитная структура приложения с отсутствием явных границ между компонентами затрудняет понимание архитектуры, увеличивает связность и усложняет поддержку.

Цель:

- Создать четкое разделение ответственности между модулями
- Обеспечить compile-time проверку архитектурных границ
- Улучшить поддерживаемость через логическое группирование кода
- Подготовить базу для возможного выделения модулей в микросервисы

Реализация: Реструктурировать приложение в модульный монолит с Spring Modulith, разделив код на модули с явными границами и публичными API.

Результаты

Выполнена полная реструктуризация монолитного приложения в модульный монолит с использованием Spring Modulith 2.0.1. Приложение разделено на модули с явными границами и зависимостями, что обеспечивает лучшую поддерживаемость, тестируемость и понимание архитектуры системы.

Структура модулей:

- shared — общие компоненты, доступные всем модулям
 - common — базовые сущности, мапперы, утилиты
 - module — абстракции стационарных модулей
 - resource — работа с ресурсами
 - user — пользователи и роли
- security — аутентификация и авторизация
- trade — торговая система

- dock — модуль стыковки
- storage — управление хранилищем
- production — производственные мощности

Принесённая ценность:

- Четкие границы модулей: Невозможность случайного нарушения архитектуры, явные контракты между модулями
- Улучшенная поддерживаемость: Логическое группирование связанного кода, легче найти нужный код, изолированное тестирование модулей
- Масштабируемость: Возможность выделения модулей в микросервисы, независимая разработка команд над модулями
- Документированная архитектура: Spring Modulith генерирует диаграммы модулей, автоматическая проверка архитектурных правил

3.2. Оптимизация конфигурации Docker

[Pull request](#)

Задача

Проблема: Сложный процесс локальной сборки и запуска, требующий установки дополнительных инструментов (Java, Gradle), что замедляет onboarding и усложняет демонстрацию проекта.

Цель:

- Обеспечить возможность запуска всего стека одной командой `docker-compose up`
- Устранить необходимость локальной установки Java/Kotlin/Gradle
- Упростить onboarding новых разработчиков
- Обеспечить единообразие окружений

Реализация: Создать оптимизированную Docker-конфигурацию с multi-stage Dockerfile и docker-compose для запуска полного стека (БД, миграции, приложение).

Результаты

Создана оптимизированная Docker-конфигурация с multi-stage сборкой и полным стеком приложения. Это устраняет необходимость локальной установки инструментов разработки и позволяет запустить всё приложение одной командой.

Что было сделано:

- Создан multi-stage Dockerfile:
 - Build stage: Использует `gradle:9.2.1-jdk21` для сборки приложения с кешированием зависимостей
 - Runtime stage: Использует `eclipse-temurin:21-jre-alpine` для минимального production образа

- Оптимизация слоёв Docker для максимального использования кэша
- Создание директории логов с правильными правами доступа
- Запуск приложения от непривилегированного пользователя для безопасности
- Создан `docker-compose-app.yml` для запуска полного стека:
 - PostgreSQL с health check
 - Flyway для миграций с зависимостью от готовности БД
 - Application с зависимостью от завершения миграций
 - Volume mapping для логов (`./logs:/app/logs`) — логи доступны в директории проекта
 - Конфигурация через переменные окружения
 - Изолированная Docker-сеть для всех сервисов
- Создан `.dockerignore` для ускорения сборки и уменьшения размера контекста

Принесённая ценность:

- Простой onboarding: Новый разработчик может запустить всё приложение командой `docker-compose -f docker-compose-app.yml up`
- Не требуется локальная установка: Не нужно устанавливать Java, Kotlin, Gradle на локальную машину
- Инкрементальная сборка: Gradle кэширует зависимости, Docker кэширует слои — пересборка только изменённого кода
- Единообразие окружений: Одинаковое окружение для разработки, тестирования и демонстрации
- Доступность логов: Логи приложения записываются в `./logs/` директорию проекта и доступны для анализа
- Изоляция: Каждый сервис работает в своей сети с явными зависимостями

- Безопасность: Приложение запускается от непривилегированного пользователя, минимальный runtime образ
- Production-ready: Multi-stage сборка создаёт оптимизированный образ без build-инструментов

3.3. Актуализация документации после рефакторинга

[Pull request](#)

Задача

Проблема: После проведения рефакторинга документация устаревает и не отражает текущее состояние проекта.

Цель:

- Обеспечить соответствие документации реальному состоянию кода
- Документировать новую модульную архитектуру
- Обновить инструкции по разработке и развертыванию
- Описать изменения в конфигурации и зависимостях

Реализация: Обновить всю проектную документацию (README, инструкции, API документация) в соответствии с изменениями, внесенными в ходе рефакторинга.

Результаты

Выполнена полная переработка документации проекта с разделением на краткий README и подробную Wiki-документацию на GitHub Wiki. Документация актуализирована в соответствии с текущим состоянием проекта после всех этапов рефакторинга.

Что было сделано:

- README
 - Создан лаконичный README.md на русском языке:
 - Ссылка на Wiki в начале документа
 - Упрощённый раздел технологий (список вместо категорий)
 - Новый раздел Scale с метриками системы (19 таблиц БД, 62 эндпоинта, 9 модулей)
 - Краткое описание предметной области

- Раздел Архитектура с описанием модулей (компонентные модули станции, другие модули, общие модули)
- Ссылка на документацию модулей на GitHub Pages
- Два варианта локального запуска (инфраструктура в Docker + приложение локально; всё в Docker)
- Создан README_EN.md (английская версия) с идентичной структурой
- GitHub Wiki
 - Создана полная структура документации на GitHub Wiki:
 - [Главная страница Wiki](#) / [Wiki Home](#) — навигация по документации
 - [Предметная область](#) / [Domain](#) — подробное описание предметной области с оглавлением
 - [Схема базы данных](#) / [Database Schema](#) — схема базы данных с группировкой таблиц по модулям
 - [Проект рефакторинга 2025](#) / [Refactoring Project 2025](#) — проект рефакторинга с планом и результатами

Структура Wiki-страниц

- Двухязычность: Все страницы доступны на русском и английском с перекрёстными ссылками
- Навигация: Ссылки на альтернативную языковую версию в формате [EN version](page) / [RU version](page)
- Оглавление: Страницы Domain содержат подробное оглавление для навигации по разделам
- GitHub Pages: Все диаграммы ссылаются на <https://killreal777.github.io/is-project/>

Принесённая ценность:

- Чёткое разделение: Краткая информация в README, подробная документация на GitHub Wiki
- Доступность: Wiki доступна всем пользователям GitHub без клонирования репозитория
- Метрики системы: Явное указание масштаба проекта (19 таблиц, 62 эндпоинта, 9 модулей)
- Двуязычность: Вся документация доступна на русском и английском языках
- Централизованная навигация: Главная страница Wiki с оглавлением обеспечивает удобный доступ
- Актуальность: Документация точно соответствует текущему состоянию кода и БД
- Согласованность с архитектурой: Группировка таблиц БД по модулям Modulith
- Доступность материалов: Диаграммы и API-документация размещены на GitHub Pages
- Профессиональный вид: Структурированная документация повышает качество представления проекта

Презентация

- смотреть видео “Демонстрация_работающего_приложения”
- смотреть видео “Обзор_выполненной_работы_код,_документация”